

DESIGN AND IMPLEMENTATION OF A 12-SERVO QUADRUPED SPIDER ROBOT WITH ARDUINO UNO

Semester Project Report

Robotics Sensors and Actuators

Submitted by:

Ahmed Raza (70177927)

Abdul Rehman (70150542)

Ubaidullah (70177858)

Muhammad Saeed (70178270)

Abstract

This semester project report details the design, wiring, and programming of a 12-degree-of-freedom (12-DOF) quadruped spider robot driven by an Arduino Uno microcontroller. The robot integrates twelve high-torque MG90s metal-gear micro servos. Wireless communication using an HC-05 Bluetooth module communicating with the mobile Dabble Application (Gamepad Module) over an asynchronous Software Serial protocol (URAT). For obstacles and safety, an HC-SR04 ultrasonic sensor is mounted to track forward obstacle distance. A high-efficiency XL-4016 step-down buck converter regulates power supply distribution. Locomotion is governed by an alternating diagonal-pair gait matrix. Physical tests verified that a shared, common-ground plane successfully eliminates signal noise and prevents controller brownouts, yielding a stable, responsive multi-legged robotics platform.

1. Introduction & Project Overview

This project focuses on the design and development of a four-legged spider robot that can move across different types of surfaces. Unlike wheeled robots, legged robots are better at handling uneven ground and obstacles. The robot is built using an Arduino Uno and uses twelve servo motors to control the movement of its legs. An ultrasonic sensor helps the robot detect obstacles, while a Bluetooth module allows wireless control from a smartphone gamepad. The project combines concepts from robotics, sensors, actuators, and embedded systems to create a functional and interactive quadruped robot.

2. System Design & Hardware Core

The robot has a main body that holds all the electronic components and four legs for movement. The legs are labeled L1 (Front Left), L2 (Front Right), L3 (Back Left), and L4 (Back Right). Each leg uses three servo motors to control its motion.

- **Coxa servo** moves the leg forward and backward.
- **Femur servo** lifts and lowers the leg.
- **Tibia servo** controls the lower part of the leg and helps it stay in contact with the ground.

These three joints work together to produce smooth and coordinated walking movements.

Hardware Components:

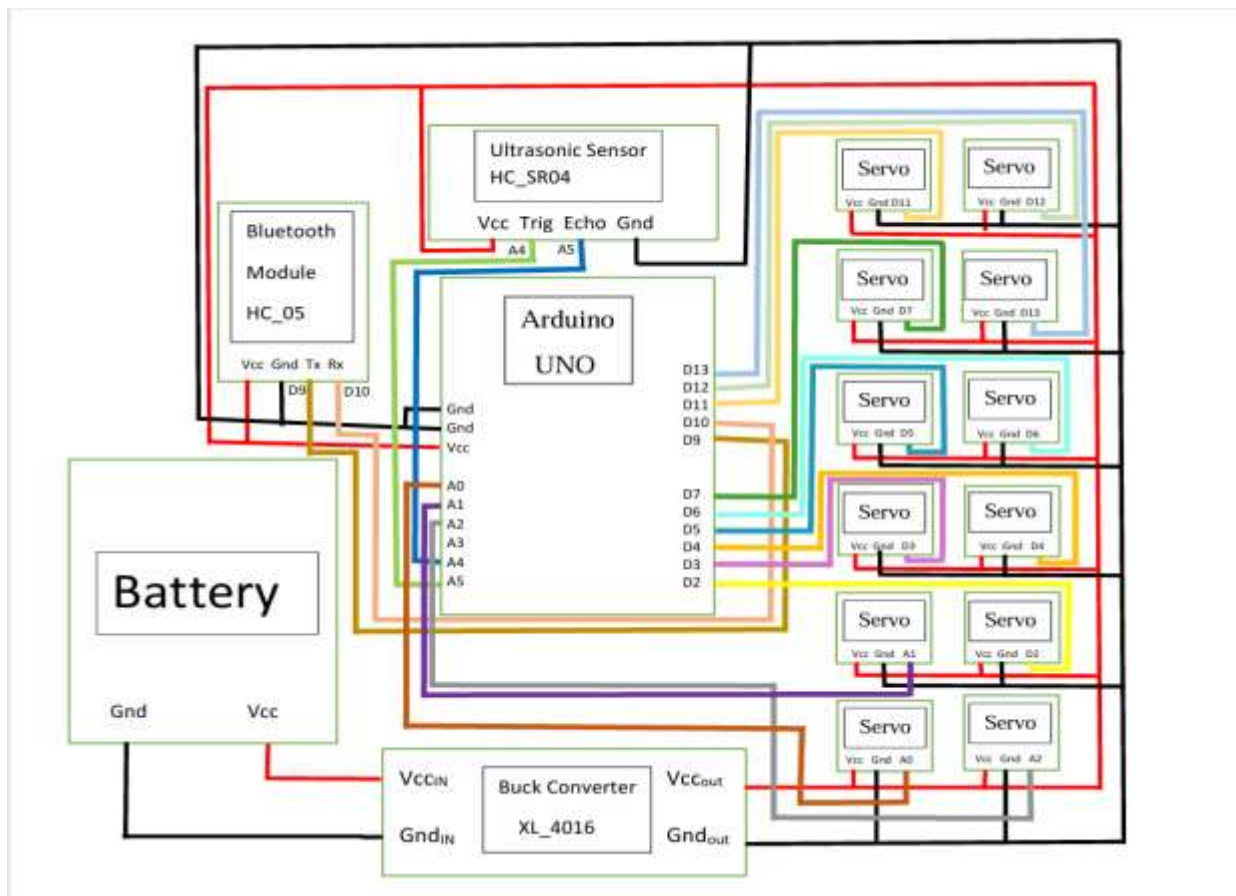
- Microcontroller Core: Arduino Uno (ATmega328P running at 16 MHz).
- Actuators: 12 x MG90s Micro Servos (Metal gear train, 2.2 kg.cm torque capacity).
- Communication Module: HC-05 Bluetooth Module (Configured via UART SoftwareSerial).
- Sensor: HC-SR04 Ultrasonic Sensor (40 kHz acoustic ranging).
- Power Supply Regulator: XL-4016 DC-DC Buck Converter (Max 8A continuous rating).

3. Circuit Diagram and Schematic Explanation

The circuit of the quadruped spider robot is centered around the Arduino Uno, which acts as the main controller for all components. Twelve servo motors are connected to the Arduino through PWM-capable digital and analog pins to control the movement of the robot's four legs.

An HC-05 Bluetooth module is connected to pins D9 and D10, allowing wireless communication between the robot and a smartphone. Commands sent from the smartphone are received by the Arduino and translated into movement instructions. An HC-SR04 ultrasonic sensor is connected to pins A4 and A5. The sensor measures the distance to nearby objects and provides obstacle detection capability. All servos, the Bluetooth module, and the ultrasonic sensor are powered using an external buck converter. Since the servo motors require more current than the Arduino can safely provide, the buck converter supplies the necessary power. The ground of the buck converter is connected to the Arduino ground to establish a common reference and ensure reliable operation.

The overall circuit enables the Arduino Uno to control locomotion, receive wireless commands, and process sensor data, allowing the robot to move and interact with its environment effectively.



4. Pin Configuration Matrix

The table below maps the complete electrical layout of the Arduino Uno to prevent signal conflicts and clearly log structural connections:

Hardware Component Module	Module Pin Type	Arduino Uno Pin	Logic Interface Type
HC-05 Bluetooth Module	Transmitter (TX)	Digital Pin D9	Software Serial RX (Input)
HC-05 Bluetooth Module	Receiver (RX)	Digital Pin D10	Software Serial TX (Output)
HC-SR04 Sonar Sensor	Echo Return Signal	Analog Pin A4	Digital Input (Pulse Tracking)
HC-SR04 Sonar Sensor	Trigger Pulse	Analog Pin A5	Digital Output (High-Speed Edge)
Leg 1 (Front Left)	Coxa / Femur / Tibia	D2, D3, D4	Digital Pins
Leg 2 (Front Right)	Coxa / Femur / Tibia	D5, D6, D7	Digital Pins
Leg 3 (Back Left)	Coxa / Femur / Tibia	A0, A1, A2	Analog Pins
Leg 4 (Back Right)	Coxa / Femur / Tibia	D11, D12, D13	Digital Pins

5. Software Architecture & Design

The robot's software is written in C++ using the Arduino IDE. The Arduino Uno acts as the main controller and continuously runs a loop that manages all robot functions.

The program receives commands from a smartphone through the HC-05 Bluetooth module and processes them in real time. Based on the received command, the Arduino controls the servo motors to perform actions such as moving forward, moving backward, turning left, turning right, obstacle avoiding or standing still.

```
#include <Servo.h>
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include <Dabble.h>
#include <SoftwareSerial.h>
SoftwareSerial BT (9,10);

// ===== Servos =====
```

```

Servo L1C, L1F, L1T;
Servo L2C, L2F, L2T;
Servo L3C, L3F, L3T;
Servo L4C, L4F, L4T;

// Ultrasonic
const int trigPin = A5;
const int echoPin = A4;
bool obstacleMode = false;

// =====
// DISTANCE FUNCTION
// =====

float getDistance() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH, 30000);
    float distance = duration * 0.0343 / 2.0;
    Serial.println(distance);
    delay(1000);
    return distance;
}

void stand() {
    // Coxa
    L1C.write(70);
    L2C.write(110);
    L3C.write(110);
    L4C.write(70);

    // Femur
    L1F.write(70);
    L2F.write(120);
    L3F.write(120);
    L4F.write(70);

    // Tibia
    L1T.write(60);
    L2T.write(120);
    L3T.write(120);
    L4T.write(60);
    delay(500);
}

```

```
}  
void forwardStep() {  
  // Front Left + Back Right  
    L1F.write(50);  
    L4F.write(50);  
    L1C.write(55);  
    L4C.write(85);  
    delay(50);  
    L1F.write(70);  
    L4F.write(70);  
    delay(50);  
    L1C.write(70);  
    L4C.write(70);  
    delay(50);  
  // Front Right + Back Left  
    L2F.write(140);  
    L3F.write(140);  
    delay(50);  
    L2C.write(125);  
    L3C.write(95);  
    delay(50);  
    L2F.write(120);  
    L3F.write(120);  
    delay(50);  
    L2C.write(110);  
    L3C.write(110);  
    delay(50);  
}  
void backwardStep() {  
    L1F.write(50);  
    L4F.write(50);  
    delay(50);  
    L1C.write(85);  
    L4C.write(55);  
    delay(50);  
    L1F.write(70);  
    L4F.write(70);  
    delay(50);  
    L1C.write(70);  
    L4C.write(70);
```

```
    delay(50);
    L2F.write(140);
    L3F.write(140);
    delay(50);
    L2C.write(95);
    L3C.write(125);

    delay(50);
    L2F.write(120);
    L3F.write(120);
    delay(50);
    L2C.write(110);
    L3C.write(110);
    delay(50);
}

void turnLeft() {
    L1F.write(50);
    L4F.write(50);
    delay(50);
    L1C.write(85);
    L4C.write(85);
    delay(80);
    L1F.write(70);
    L4F.write(70);
    delay(50);
    L2F.write(140);
    L3F.write(140);
    delay(50);
    L2C.write(125);
    L3C.write(125);
    delay(80);
    L2F.write(120);
    L3F.write(120);
    delay(50);
    stand();
}

void turnRight() {
    L1F.write(50);
    L4F.write(50);
    delay(50);
```

```

    L1C.write(55);
    L4C.write(55);
    delay(80);
    L1F.write(70);
    L4F.write(70);
    delay(50);
    L2F.write(140);
    L3F.write(140);
    delay(50);

    L2C.write(95);
    L3C.write(95);
    delay(80);
    L2F.write(120);
    L3F.write(120);
    delay(50);
    stand();
}

void salute() {
    L2F.write(150);
    L2T.write(0);
    delay(400);
    for(int i = 0; i < 2; i++)
    {
        L2C.write(90); delay(250);
        L2C.write(130); delay(250);
    }
    L2C.write(110);
    L2F.write(120);
    L2T.write(120);
    delay(300);
    stand();
}

void setup(){
    BT.begin(38400);
    Dabble.begin(BT);
    Serial.begin(9600);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    L1C.attach(2); L1F.attach(3); L1T.attach(4);

```

```

    L2C.attach(5); L2F.attach(6); L2T.attach(7);
    L3C.attach(A0); L3F.attach(A1); L3T.attach(A2);
    L4C.attach(11); L4F.attach(12); L4T.attach(13);
    stand();
}

void loop() {
    Dabble.processInput();
    if(GamePad.isCirclePressed()) {obstacleMode = true;}
    if(GamePad.isCrossPressed()) {obstacleMode = false;}
    if(obstacleMode) {

        float d = getDistance();
        if (d <= 20){
            backwardStep(); delay(200);
            turnRight(); delay(200);
        }
        else {forwardStep();}
    }
    If (GamePad.isUpPressed()) {forwardStep();}
    else if (GamePad.isDownPressed()) {backwardStep();}
    else if (GamePad.isLeftPressed()) {turnLeft();}
    else if (GamePad.isRightPressed()) {turnRight();}
    else if (GamePad.isSquarePressed()) {stand();}
    else if (GamePad.isTrianglePressed()) {salute();}
}

```

6. Dabble Gamepad Control Integration

The robot is controlled wirelessly using the Dabble Mobile App (Gamepad mode). Instead of handling complicated serial commands, the system uses a simple built-in function called `Dabble.processInput()` to read the controls in the background.

Each button on the gamepad is linked to a specific movement or action in the code:

- **D-Pad Up:** Makes the robot move forward continuously using the `forwardStep()` function.
- **D-Pad Down:** Makes the robot move backward using the `backwardStep()` function.
- **D-Pad Left / Right:** Turns the robot left or right using `turnLeft()` and `turnRight()` functions.

- **Square Button:** Stops all movement immediately and returns the robot to a stable standing position using `stand()`.
- **Triangle Button:** Runs a special display move called `salute()`.
- **Circle / Cross Buttons:** Switches obstacle mode on or off. When it is ON, the robot works on its own; when it is OFF, it is controlled manually.

7. Conclusion

This semester project shows how mechanical design, programming, and electronics can work together using an Arduino Uno. A buck converter (XL4016) was used to step down the voltage, and all components shared a common ground. This helped solve common problems like servo shaking and unexpected resets caused by power drops.

The Dabble mobile app made the robot easy to control wirelessly using a simple gamepad-style interface. At the same time, the ultrasonic sensor allows the robot to detect obstacles and avoid them automatically.

Overall, this project provides a solid base for future development in multi-legged walking robots and improved movement control systems.